

# 4. Finite State Machines

Computer Architecture

# Executive Summary

- **Motivation:** Combinational circuits with sequential circuits allow to implement a complex design
- **Problem:** We need a procedure to transform our design idea to a circuit
- **Overview:**
  - State definitions.
  - Finite State Machine (FSM) design.
- **Conclusion:** A complex idea can be implemented as digital design unit using sequential and combinational building blocks

# Outline

Introduction

Finite State Machines

FSM Example

Conclusion

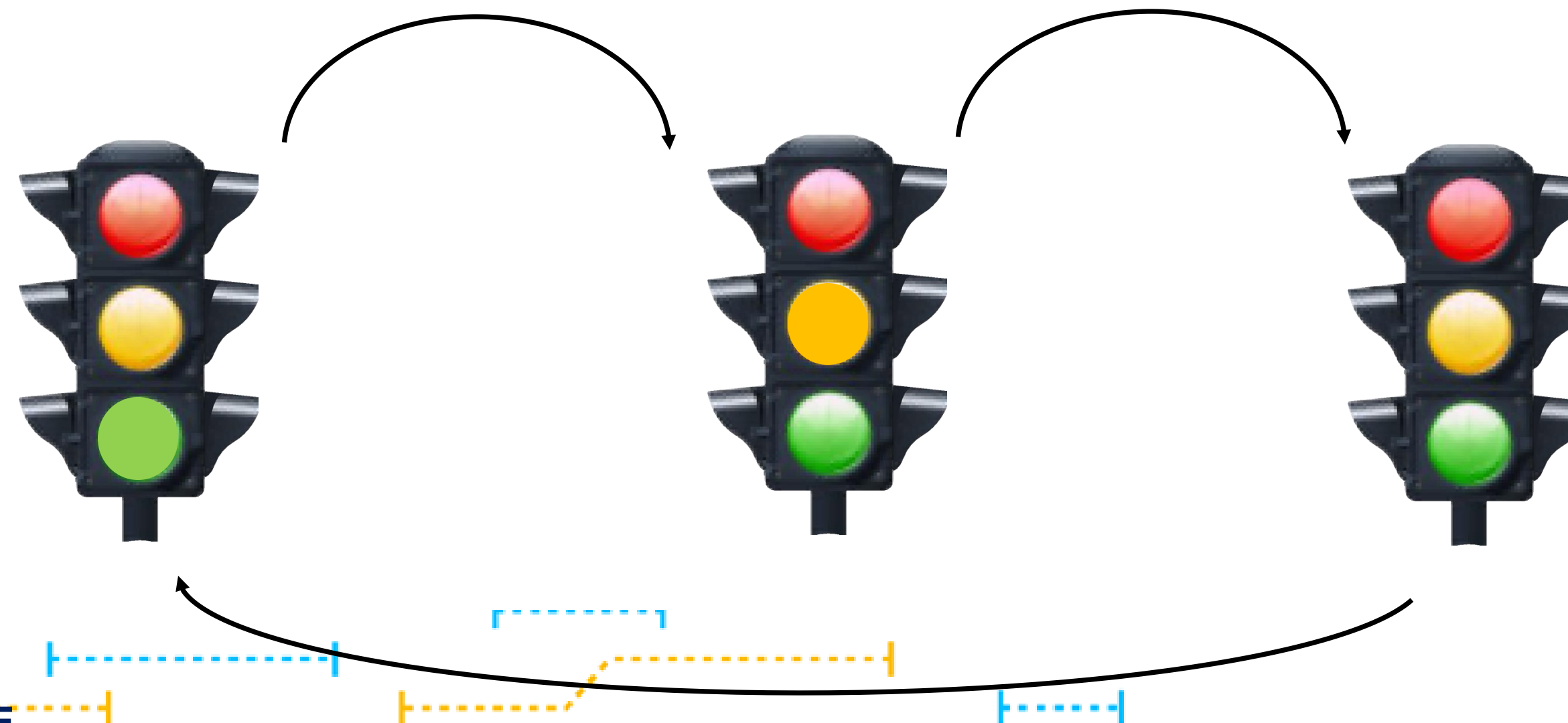


# State Definitions

- **State:**

- **In a system**, it is a **snapshot** of all the important elements (at the time of taking the snapshot).
- **In a circuit**, all the information about a **circuit** necessary to define its future behavior.
  - **State elements:** “store the state” or store one bit (e.g.: latches, flip-flops).

- **Diagram example:** a semaphore

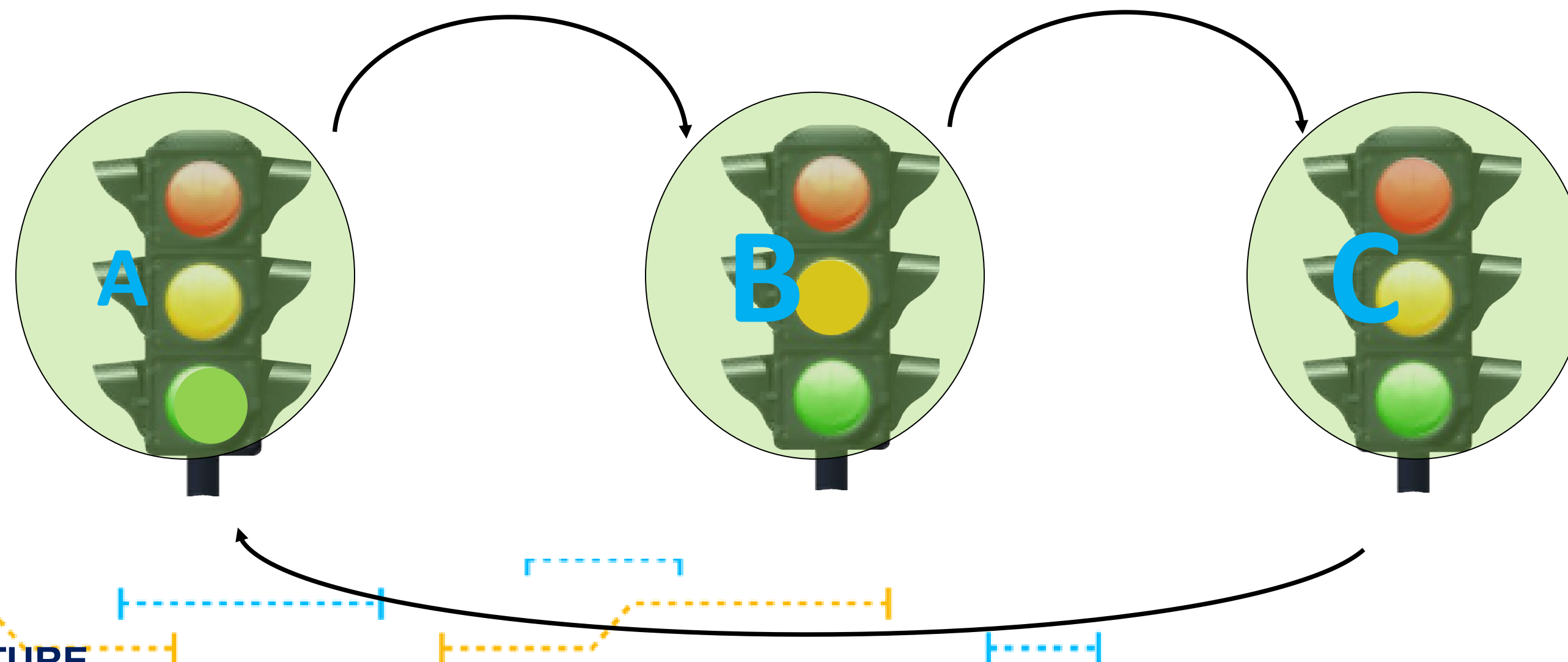


# State Definitions

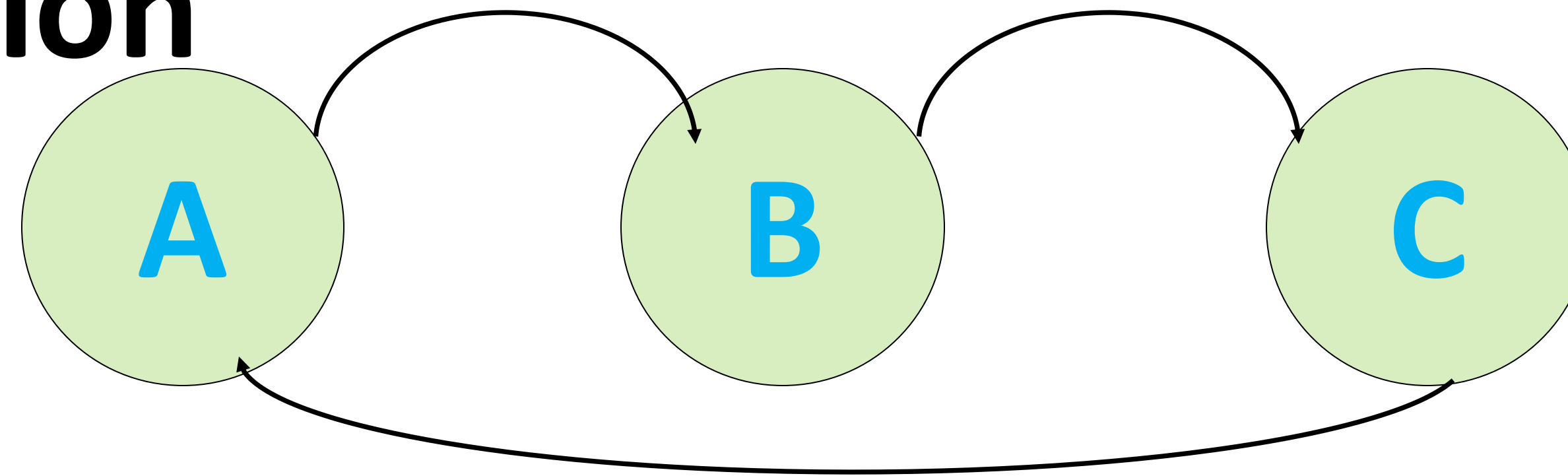
- **State:**

- **In a system**, it is a **snapshot** of all the important elements (at the time of taking the snapshot).
- **In a circuit**, all the information about a **circuit** necessary to define its future behavior.
  - **State elements:** “store the state” or store one bit (e.g.: latches, flip-flops).

- **Diagram example:** a semaphore



# State Transition



- **Clock** is a general mechanism that **triggers transition from one state to another** in a sequential circuit
  - Clock **synchronizes state changes** across many sequential circuit elements
  - Combinational logic evaluates for the length of the clock cycle

CLK: 0

- **At the start of a clock cycle** ( ), system state changes
  - **During a clock cycle, the state stays constant**
  - In the semaphore example, **we are assuming the traffic light stays in each state an equal amount of time**



# Outline

Introduction

Finite State Machines

FSM Example

Conclusion

# Finite State Machine (FSM)

- A **discrete-time model of a stateful system**
- Each state represents a **snapshot** of the system at a given time
- Can **model** a semaphore, washing machine, a computer processor, etc.
- An **FSM state diagram can visually represent**:
  1. All possible states
  2. Transition conditions
- It is defined as **finite because** it has:
  1. A **finite** number of states (snapshot)
  2. A **finite** number of external inputs
  3. A **finite** number of external outputs
  4. An explicit specification of all state transitions
  5. An explicit specification of what determines each external output value

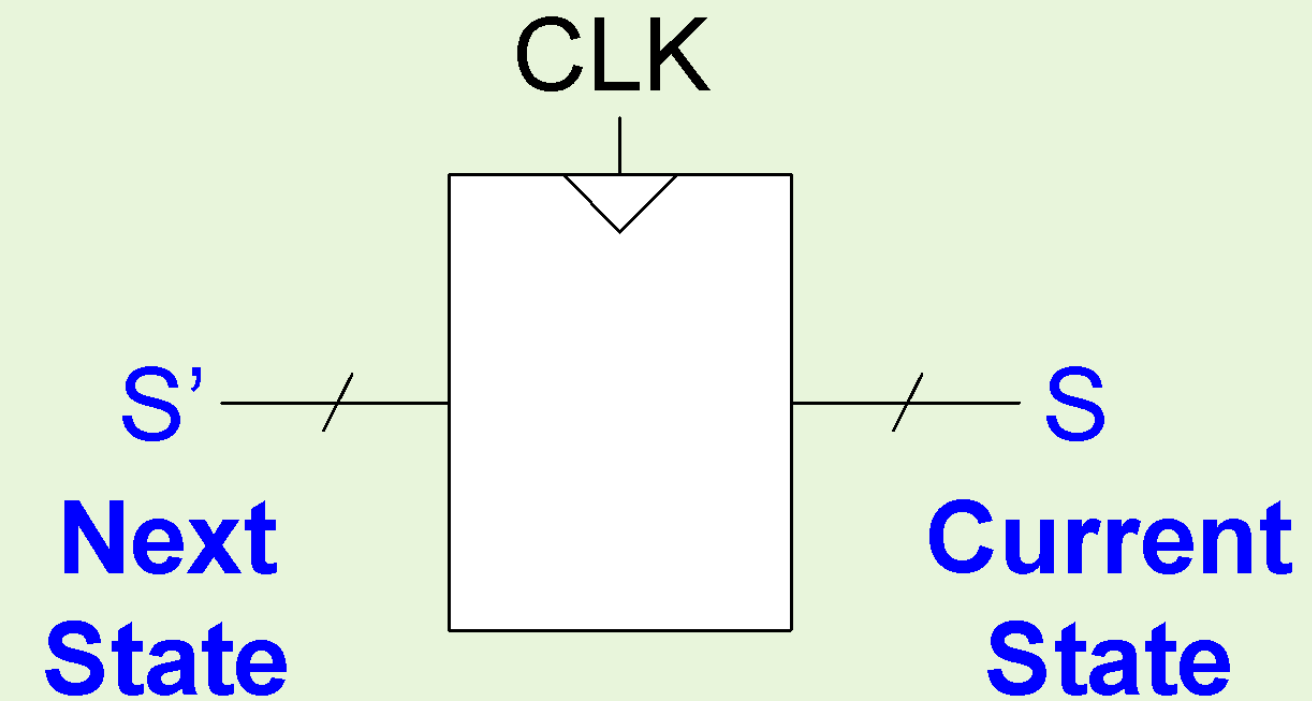


# FSM Elements

- **Sequential Circuits**

- **State register**

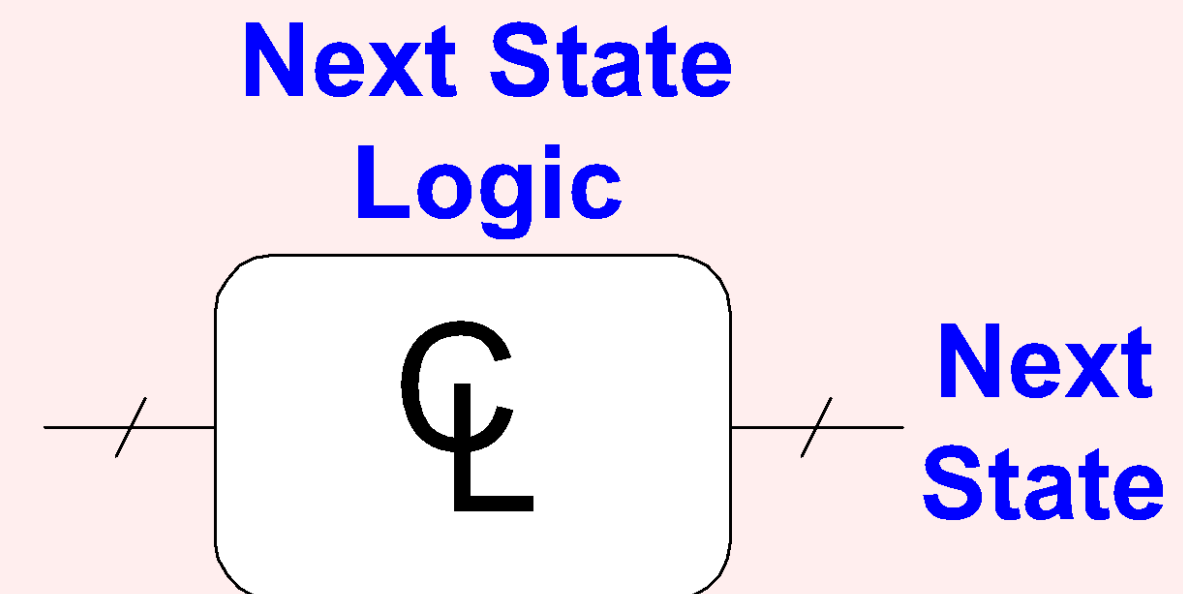
- Stores current state
    - Loads next state at clock edge



- **Combinational Circuits**

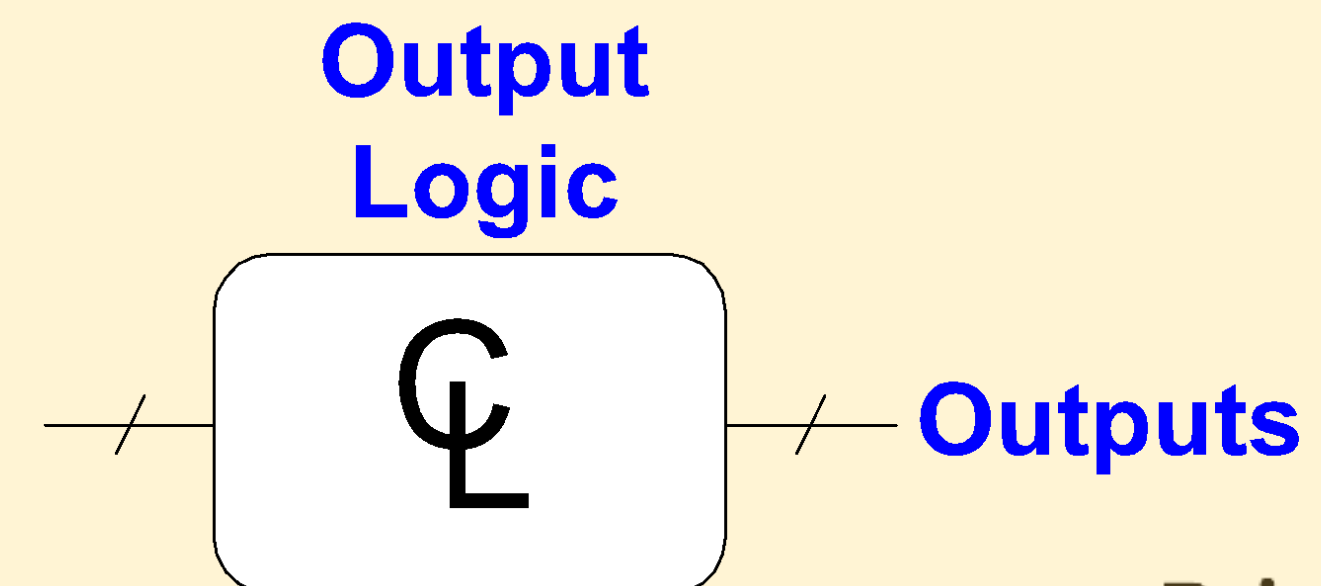
- **Next state logic:**

- Computes the next state



- **Output logic:**

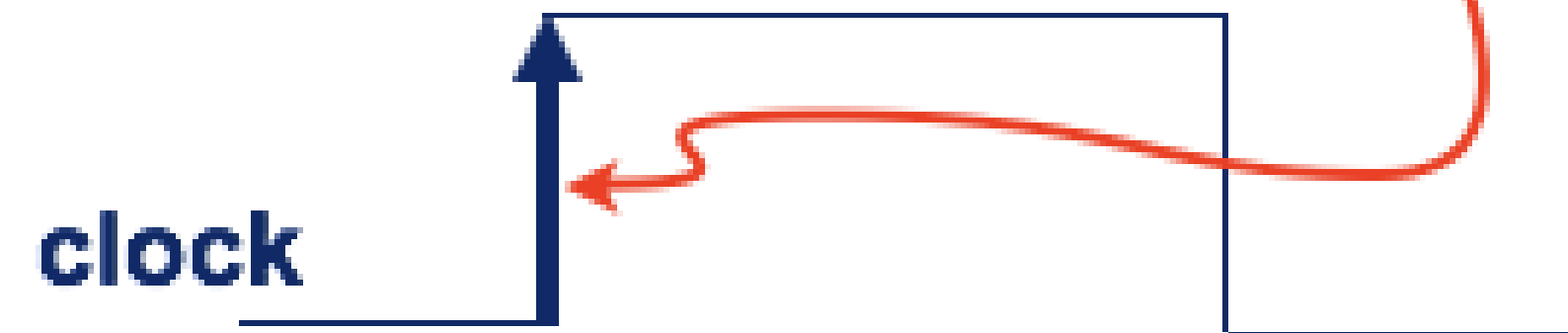
- Computes the outputs



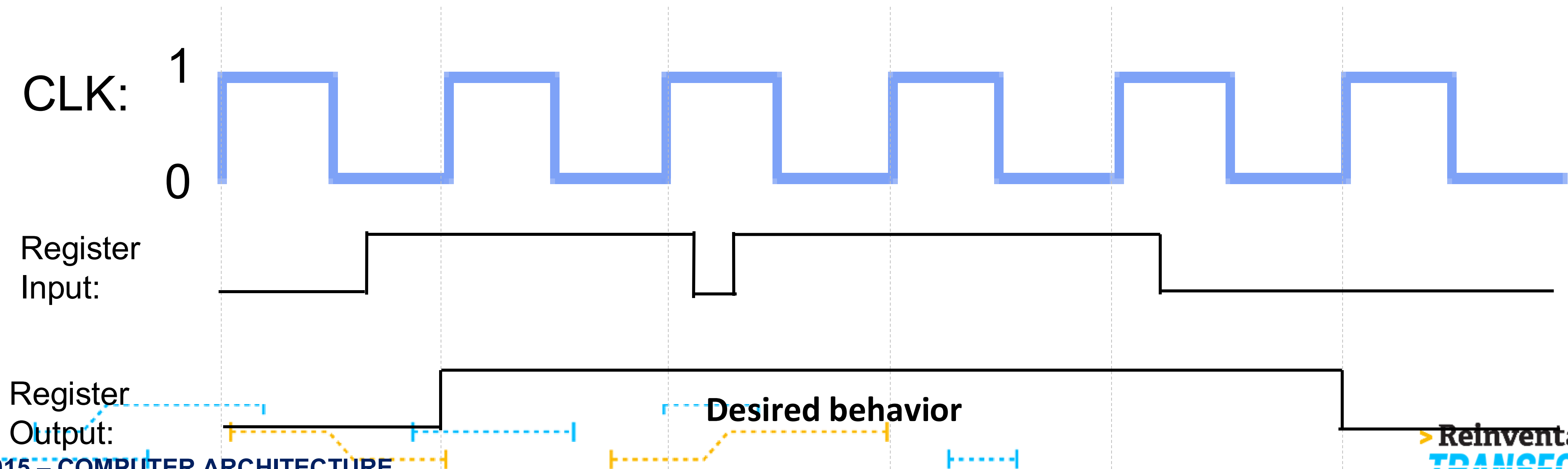
# State Register Implementation

- How can we implement a **state register**? Two properties:

- We need to store data at the **beginning** of every clock cycle  
**Clock transition determines when input is acquired**

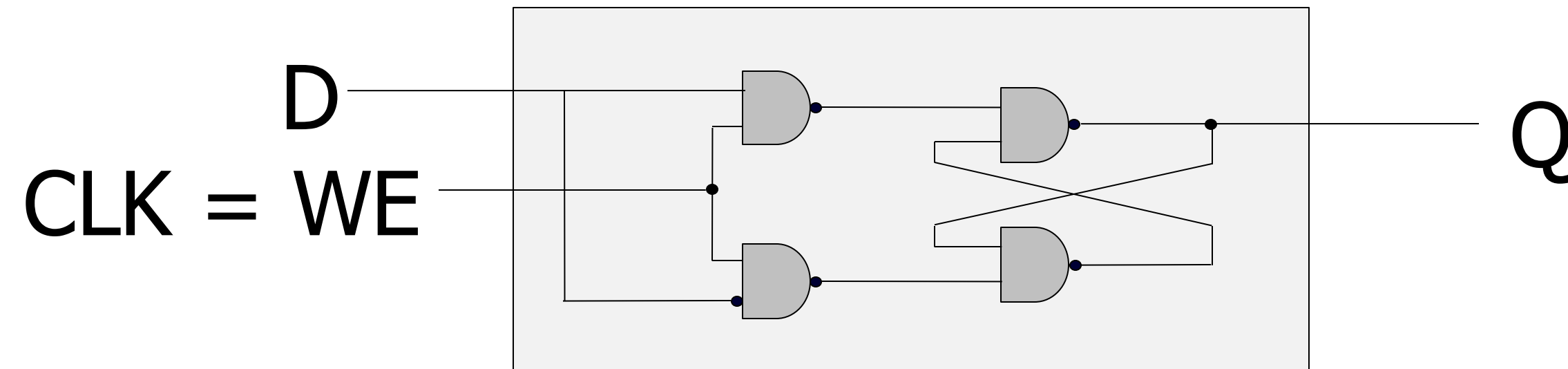


- The data must be **available** during the entire clock cycle



# Problem: State Register with Latches

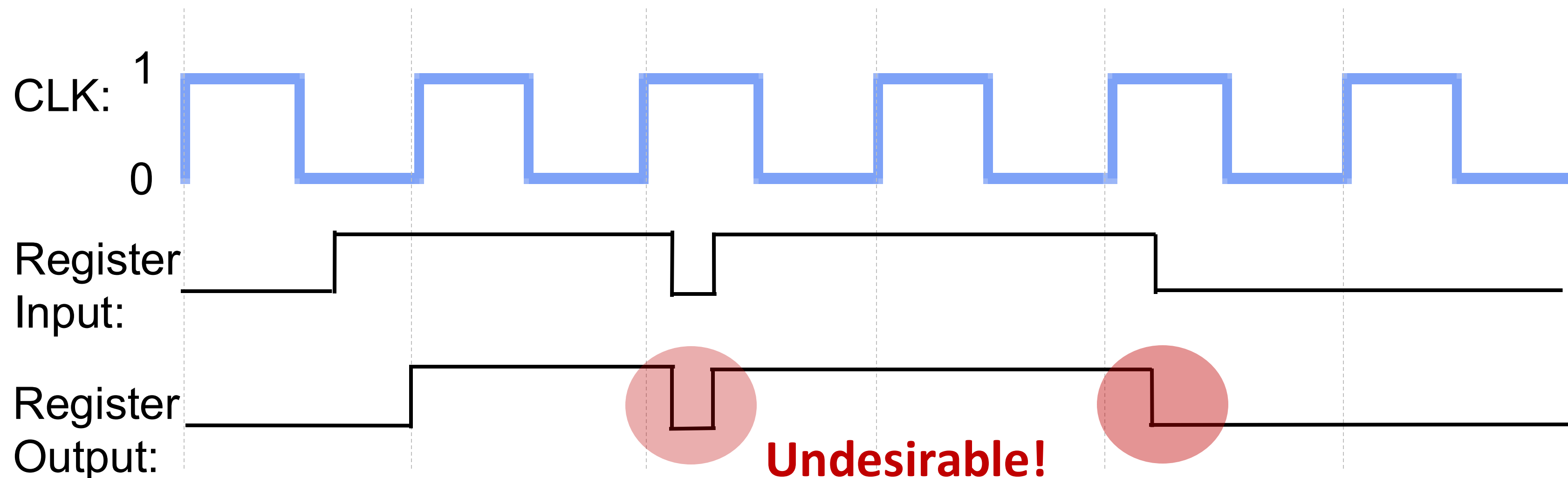
Recall the  
Gated D Latch



If we wire a clock to WE of a latch

Whenever the clock is high, the latch propagates D to Q

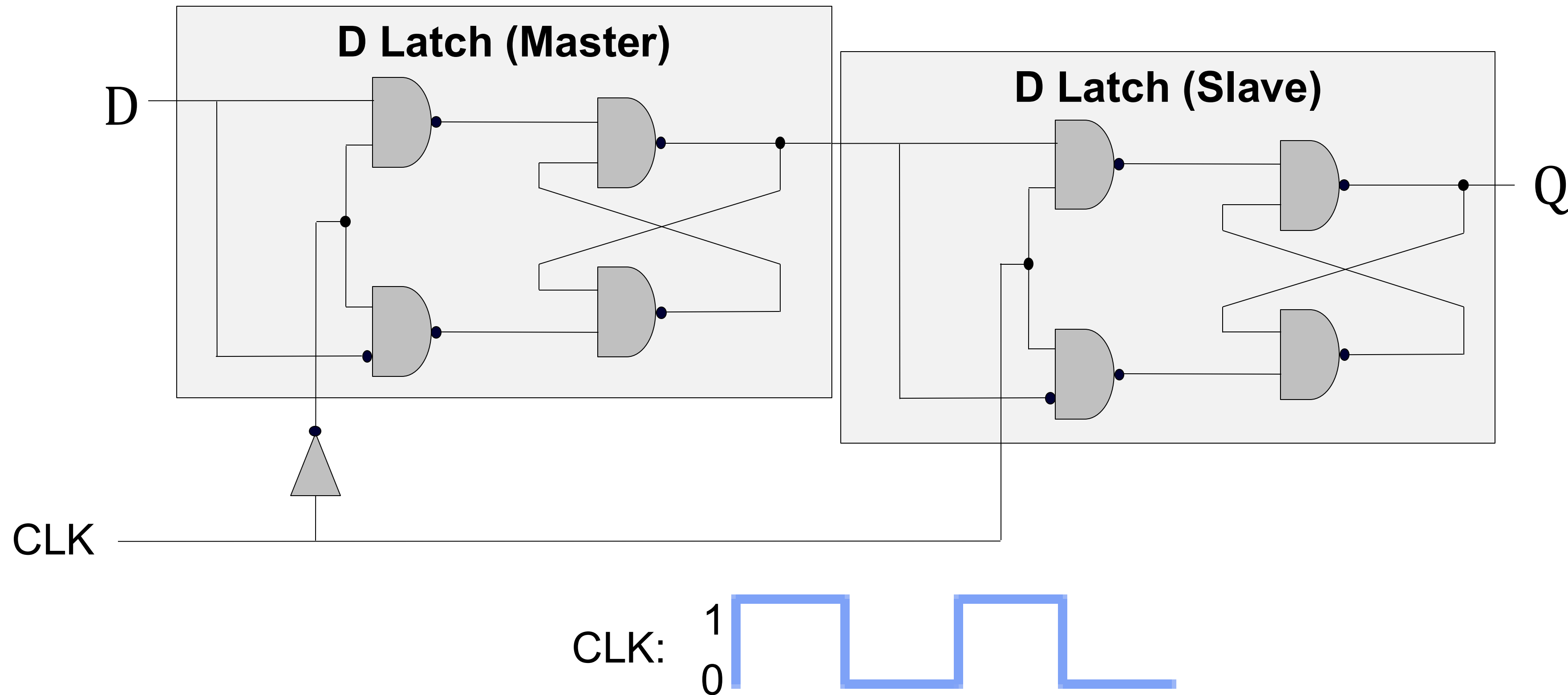
The latch is transparent





# Solution: State Register with D Flip-Flop

- 1) state change on clock edge, 2) data available for full cycle



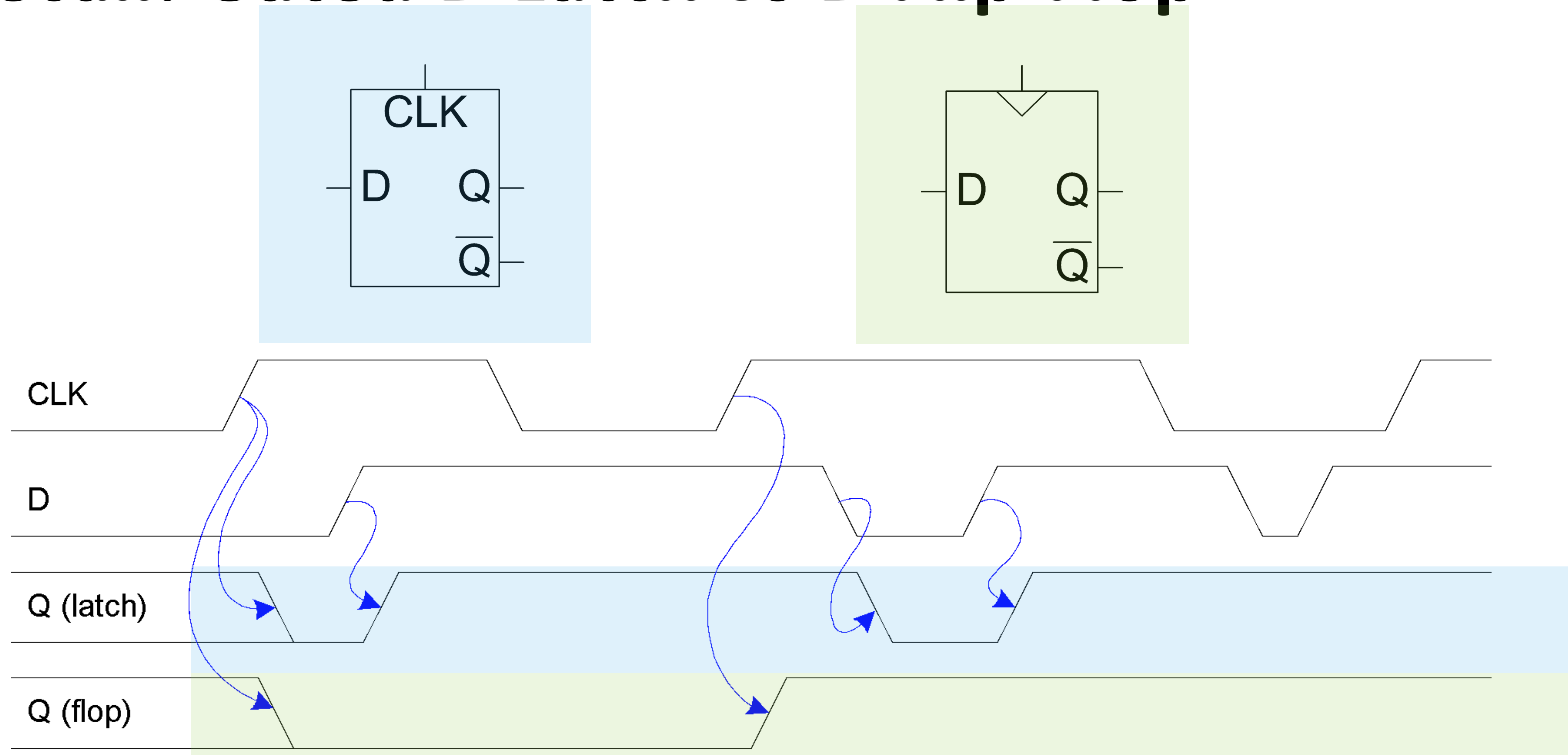
**When the clock is low**, master propagates **D** to the input of slave (Q unchanged)

**Only when the clock is high**, slave latches **D** (Q stores D)

At the rising edge of clock (clock going from 0→1), Q gets assigned D

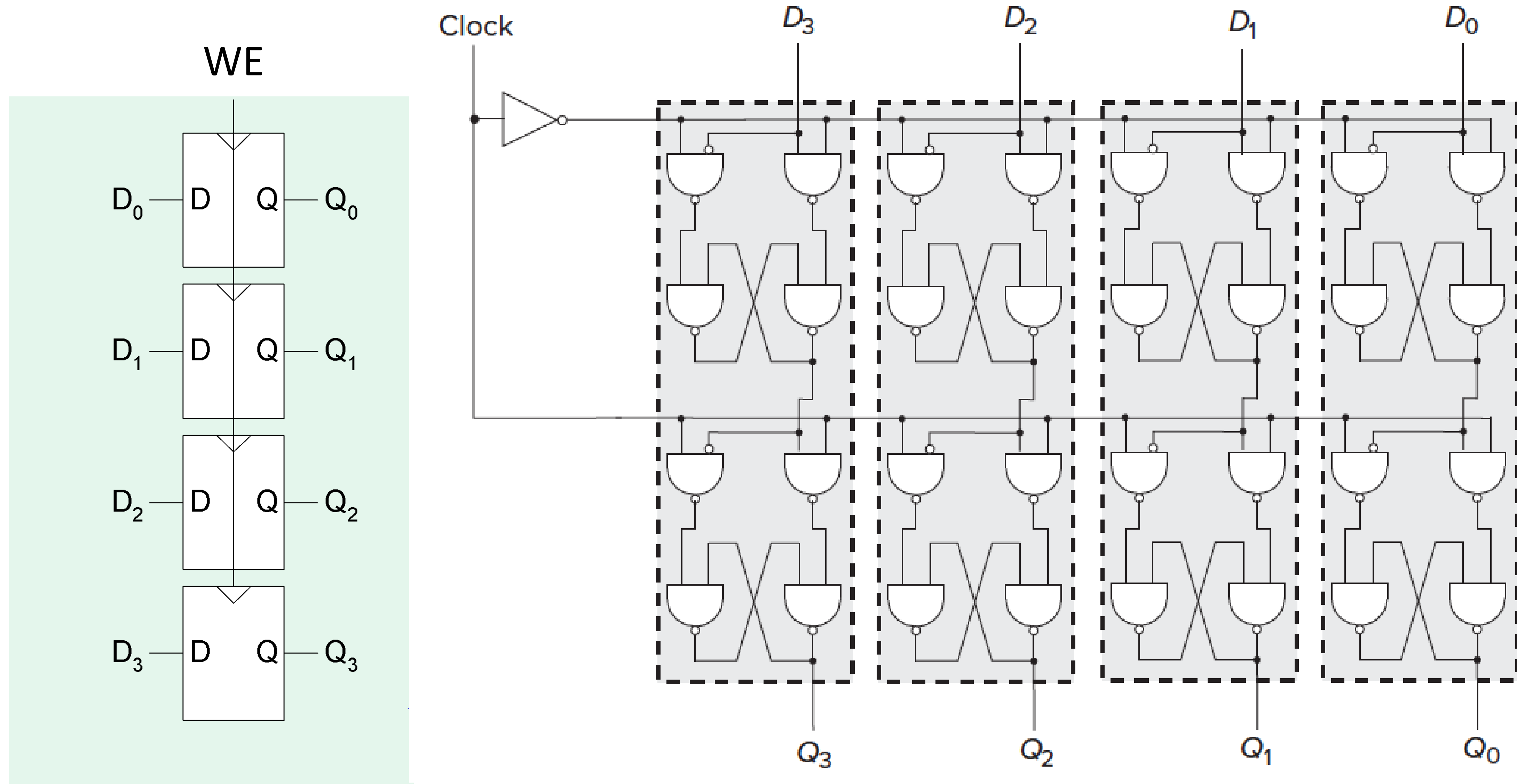


# Recall: Gated D Latch vs D Flip-Flop



**At the lab we used a clock (clk) to change the Q value using D input**

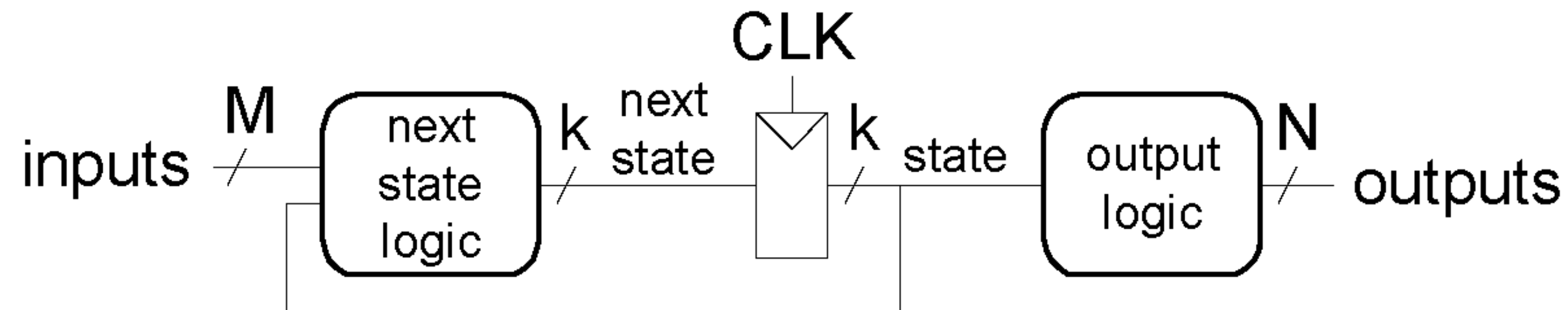
# Recall: 4-bit Register with D Flip Flop



# Types of FSM

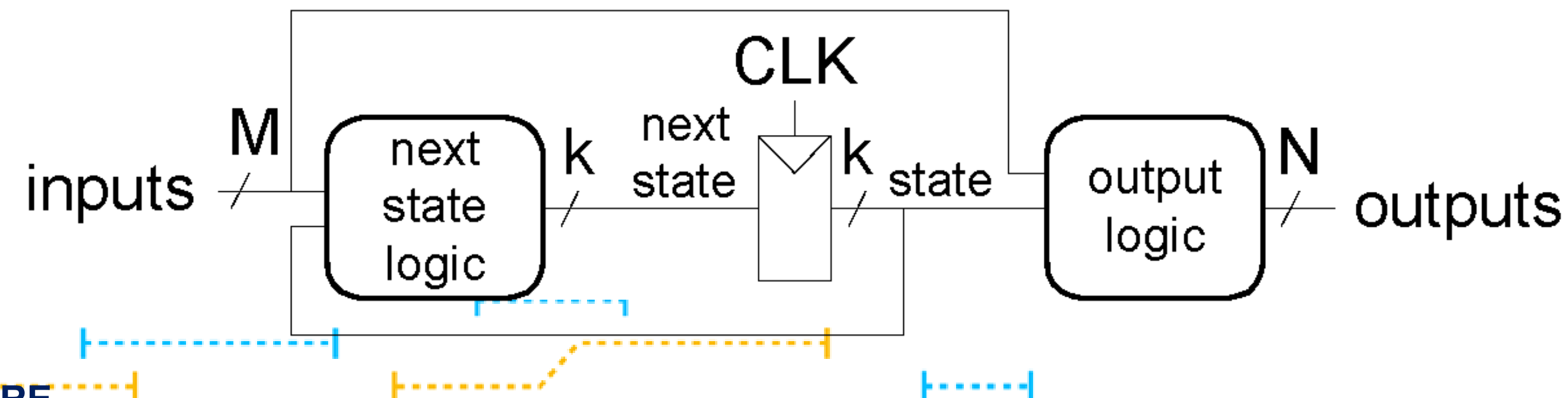
- **Next state** determined by current state and inputs
- Two types of finite state machines differ in **output logic**:
  1. **Moore FSM: outputs depend only on current state**

Moore FSM



1. **Mealy FSM: outputs depend on current state *and* inputs**

Mealy FSM





# FSM Design Procedure

1. Identify **inputs and outputs**
2. **Sketch state transition diagram**
3. **Write state transition table**
4. **Select state encodings**
5. **For Moore machine:**
  - a. Rewrite state transition table with state encodings
  - b. Write output table
5. **For a Mealy machine:**

Rewrite combined state transition and output table with state encodings
6. **Write Boolean equations for next state and output logic**
7. **Sketch the circuit schematic**



# Outline

Introduction

Finite State Machines

FSM Example

Conclusion

# FSM Exercise

- Draw a state machine that can detect when it has received the serial input sequence 01010. Use an input X and an output Z.

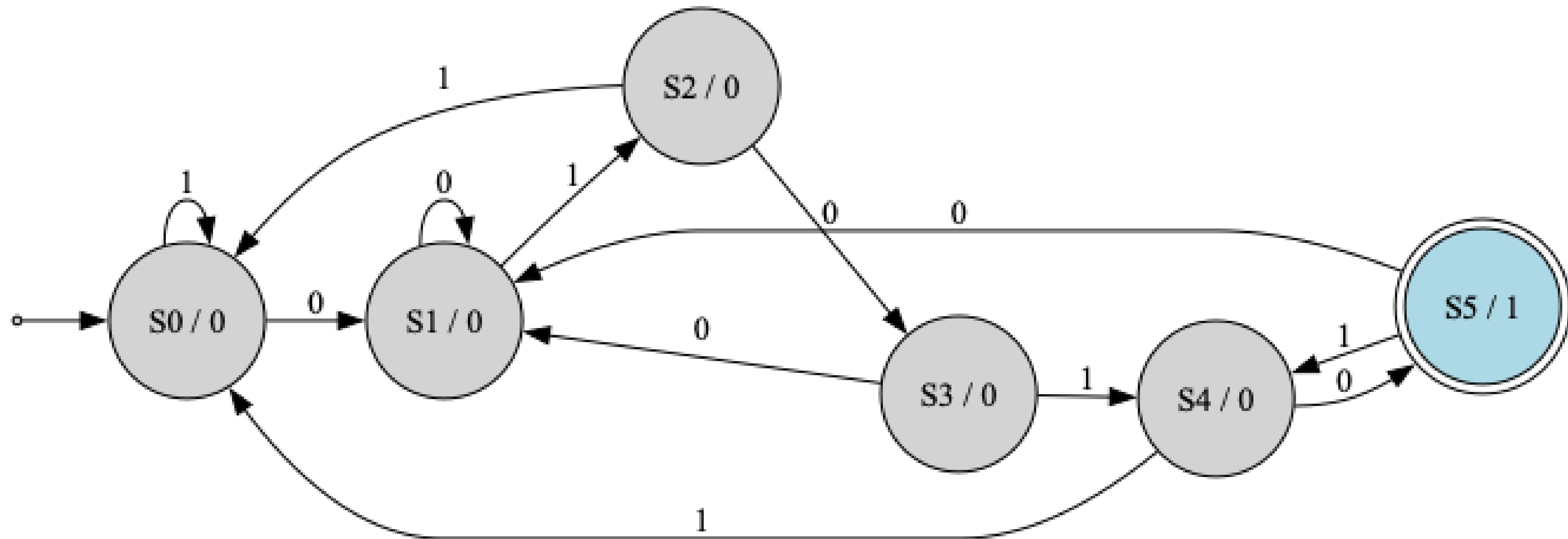
1. Draw the FSM state diagram
2. Write the Next State logic table and its equations
3. Write the Output logic table and its equations
4. Draw the FSM schematic

# FSM Black Box

- **Inputs: ?**
- **Outputs: ?**



# 1.FSM State Diagram



# 2. Next State logic

| Current State<br>(Encoding) | Input (X) | Next State<br>(Encoding) |
|-----------------------------|-----------|--------------------------|
| S0 (000)                    | 0         | S1 (001)                 |
| S0 (000)                    | 1         | S0 (000)                 |
| S1 (001)                    | 0         | S1 (001)                 |
| S1 (001)                    | 1         | S2 (010)                 |
| S2 (010)                    | 0         | S3 (011)                 |
| S2 (010)                    | 1         | S0 (000)                 |
| S3 (011)                    | 0         | S1 (001)                 |
| S3 (011)                    | 1         | S4 (100)                 |
| S4 (100)                    | 0         | S5 (101)                 |
| S4 (100)                    | 1         | S0 (000)                 |
| S5 (101)                    | 0         | S1 (001)                 |
| S5 (101)                    | 1         | S4 (100)                 |

|      |         |         |         |         |
|------|---------|---------|---------|---------|
|      | S1S0=00 | S1S0=01 | S1S0=11 | S1S0=10 |
| S2=0 | 0       | 0       | 0       | 0       |
| S2=1 | 0       | 0       | d       | d       |

|      |         |         |         |         |
|------|---------|---------|---------|---------|
|      | S1S0=00 | S1S0=01 | S1S0=11 | S1S0=10 |
| S2=0 | 0       | 1       | 0       | 0       |
| S2=1 | 0       | 0       | d       | d       |

|      |         |         |         |         |
|------|---------|---------|---------|---------|
|      | S1S0=00 | S1S0=01 | S1S0=11 | S1S0=10 |
| S2=0 | 0       | 0       | 1       | 0       |
| S2=1 | 0       | 1       | d       | d       |



## 2. Next State logic

| Current State<br>(Encoding) | Input (X) | Next State<br>(Encoding) |
|-----------------------------|-----------|--------------------------|
| S0 (000)                    | 0         | S1 (001)                 |
| S0 (000)                    | 1         | S0 (000)                 |
| S1 (001)                    | 0         | S1 (001)                 |
| S1 (001)                    | 1         | S2 (010)                 |
| S2 (010)                    | 0         | S3 (011)                 |
| S2 (010)                    | 1         | S0 (000)                 |
| S3 (011)                    | 0         | S1 (001)                 |
| S3 (011)                    | 1         | S4 (100)                 |
| S4 (100)                    | 0         | S5 (101)                 |
| S4 (100)                    | 1         | S0 (000)                 |
| S5 (101)                    | 0         | S1 (001)                 |
| S5 (101)                    | 1         | S4 (100)                 |

$$S'_2 = S_2 \overline{S_1} \overline{S_0} \overline{x} + S_0 x \overline{S_2} \overline{S_1}$$

$$S'_1 = S_0 x \overline{S_2} \overline{S_1} + S_1 \overline{S_2} \overline{S_0} \overline{x}$$

$$S'_0 = \overline{x} \overline{S_1} + \overline{x} \overline{S_2}$$

# 3.Output logic

| State (Encoding) | Output (Z) |
|------------------|------------|
| S0 (000)         | 0          |
| S1 (001)         | 0          |
| S2 (010)         | 0          |
| S3 (011)         | 0          |
| S4 (100)         | 0          |
| S5 (101)         | 1          |

|      |         |         |                  |         |
|------|---------|---------|------------------|---------|
|      | S1S0=00 | S1S0=01 | <b>Z</b> S1S0=11 | S1S0=10 |
| S2=0 | 0       | 0       | 0                | 0       |
| S2=1 | 0       | 1       | d                | d       |

$$Z = S_2 \overline{S_1} S_0$$

# 4. Schematic



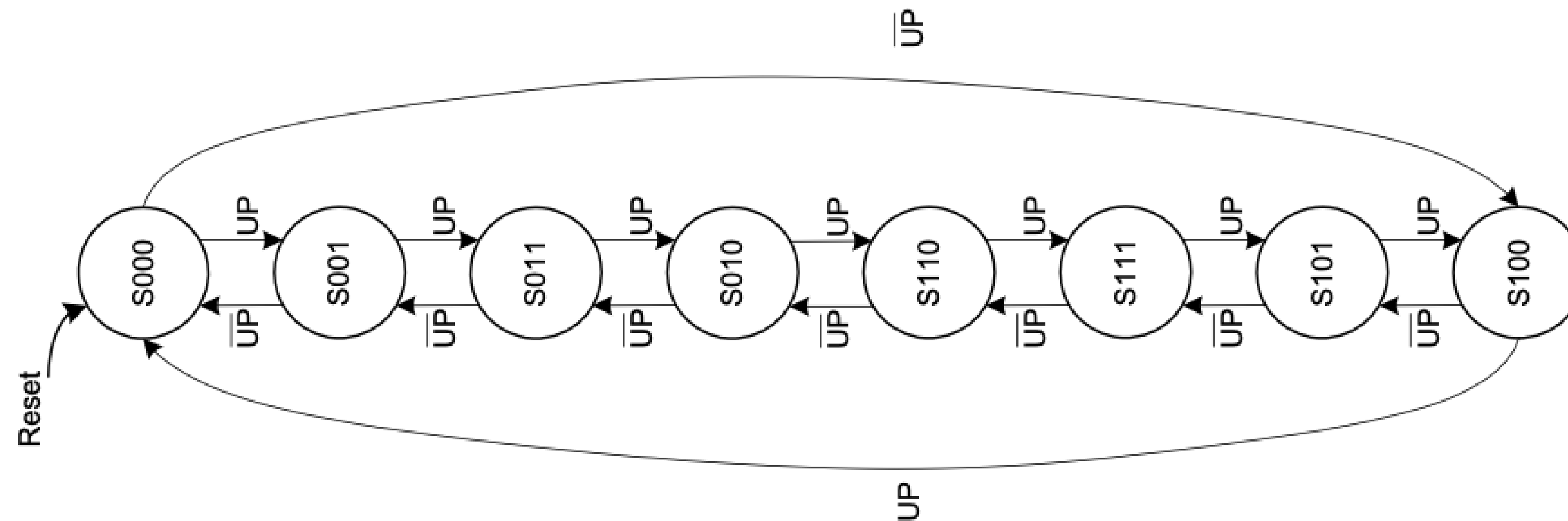
# FSM example

- **Design a** modulo 8 Gray code counter with an UP/DOWN counter. Define an UP input. If UP= 1, the counter advances to the next number. If UP= 0, the counter retreats to the previous number.
- Its output is Y, where Y=1 every time the Gray count has only one zero.

# FSM Black Box

- **Inputs: ?**
- **Outputs: ?**

# 1. Design the state diagram





## 2. Write down the next state logic

| current<br>state<br>$s_{2:0}$ | input<br>up | next state<br>$s'_{2:0}$ |
|-------------------------------|-------------|--------------------------|
| 000                           | 1           | 001                      |
| 001                           | 1           | 011                      |
| 011                           | 1           | 010                      |
| 010                           | 1           | 110                      |
| 110                           | 1           | 111                      |
| 111                           | 1           | 101                      |
| 101                           | 1           | 100                      |
| 100                           | 1           | 000                      |
| 000                           | 0           | 100                      |
| 001                           | 0           | 000                      |
| 011                           | 0           | 001                      |
| 010                           | 0           | 011                      |
| 110                           | 0           | 010                      |
| 111                           | 0           | 110                      |
| 101                           | 0           | 111                      |
| 100                           | 0           | 101                      |

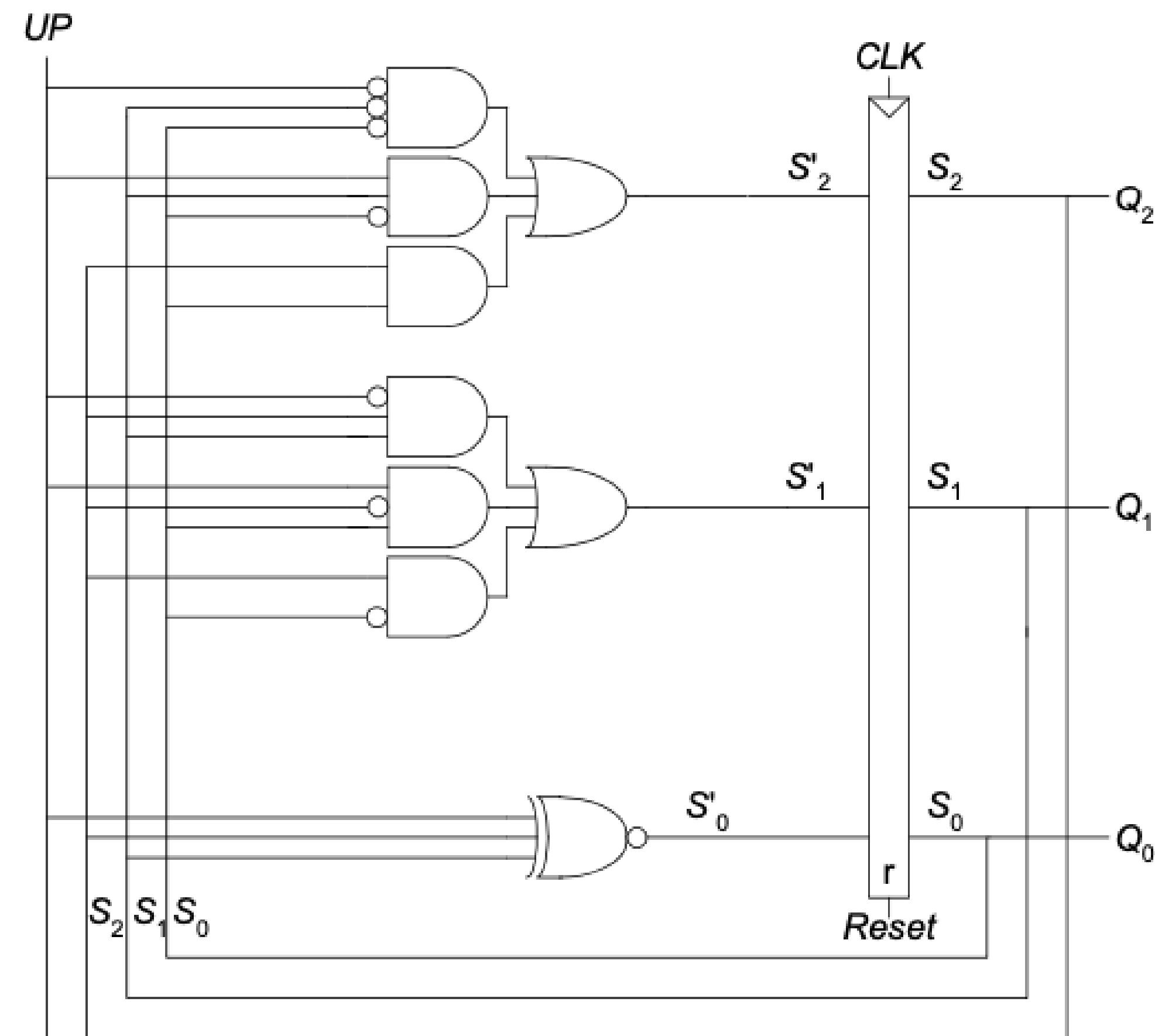
$$S'_2 = UPS_1\bar{S}_0 + \bar{U}\bar{P}\bar{S}_1\bar{S}_0 + S_2S_0$$

$$S'_1 = S_1\bar{S}_0 + UPS_2S_0 + \bar{U}\bar{P}S_2S_1$$

$$S'_0 = UP \oplus S_2 \oplus S_1$$

### 3. Write down the output logic

# 4. Draw the schematic





# Outline

Introduction

Finite State Machines

FSM Example

Conclusion



# Conclusions

- We introduced fundamentals concepts finite state machines
- We detailed the design of finite state machines.
- A sequential circuit with combinational circuit allows to implement a finite state machine.
- We conclude that complex design can be implemented as a digital circuit following design guidelines and defining state transitions.



# 4. Finite State Machines

## Computer Architecture